

RA3. Condicionals, bucles i excepcions

A2 - Bucles

Índex

1 Estructures de repetició

2 El bucle FOR

3 El bucle WHILE

4 El bucle DOWHILE

5 Break i Continue

6 Quan utilitzo cada tipus de bucles?

1.- Estructures de repetició

- Fins ara, un cop una instrucció ha estat executada, ja no es tornarà a executar mai més.

Les estructures de repetició o iteratives permeten repetir una mateixa seqüència d'instruccions diverses vegades, mentre es compleixi una certa condició.

1.- Estructures de repetició

- En el seu aspecte general, tenen molt de semblant a una estructura de selecció:
  - Hi ha una sentència especial que cal escriure al codi font: for, while o do-while.
  - Unida a una condició lògica.
  - Un bloc de codi

- Però en aquest cas, mentre la condició lògica sigui certa, tota la seqüència d'instruccions es va executant repetidament.

1.- Estructures de repetició

- Anomenarem **bucle**, o cicle, al conjunt d'instruccions que s'ha de repetir un cert nombre de vegades, i anomenem **iteració** cada execució individual del bucle.

- Si la condició sempre és true el problema és molt més greu. Com que absolutament sempre que s'avalua si cal fer una nova iteració, la condició es compleix, el bucle no es deixa mai de repetir. El programa mai no acabarà!

    - Un bucle infinit és una seqüència d'instruccions dins d'un programa que itera indefinidament, normalment perquè s'espera que s'assoleixi una condició que mai no s'arriba a produir.

1.- Estructures de repetició

- Tenint en compte el perill que un programa s'acabi executant indefinidament, forçosament dins de tot bucle hi ha d'haver instruccions que manipulin variable el valor de les quals permeti controlar la repetició o el final del bucle. Aquestes variables s'anomenen **variables de control**.

- Quan genereu un programa, és imprescindible que el codi permeti que, en algun moment, que la variable de control canviï de valor, de manera que la condició lògica es deixi de complir. Això s'anomena **"trencar la condició".**

- Per convenció, les variables de control solen dir-se: i, j, k .

1.- Estructures de repetició

- Normalment, les variables de control dins d'un bucle es poden englobar dins d'algun d'aquests tipus de comportament:
  - Comptador: Suma una quantitat constant a cada pas (de un en un, per exemple).
  - Acumulador: Suma una quantitat variable en cada iteració (ex. suma = suma + preuProducte)
  - Semàfor o bandera: una variable que serveix com a interruptor de si cal seguir fent iteracions. Habitualment es fa servir variables booleanes.

1.- Estructures de repetició

- La variable de control té tres fases:
  - Inicialització i declaració de la variable.

    ```
    int i = 0;
    ```

  - Condició de compliment del bucle: Sol ser una expressió booleana

    i < 10

  - Condició per trencar la condició del bucle. Habitualment, augment del valor de la variable, canvi de valor de la variable...

    i = i + 1; □ i++;

2.- El bucle FOR

- En algunes situacions especials ja es coneix, a priori, la quantitat exacta de vegades que caldrà repetir el bucle.
- La sentència for permet repetir un nombre determinat de vegades un conjunt d'instruccions.

```
for (inicialització comptador; expressió booleana; increment comptador){

    //Instruccions per executar dins del bucle

}
```

2.- El bucle FOR

- Inicialització: es tracta de la inicialització d'una variable sencera que servirà com a comptador. És exactament igual que assignar un valor a una variable qualsevol (tipus identificador = valorInicial).

- Expressió booleana: es tracta de la condició lògica que indica si cal fer una nova iteració o no, igual que en la resta d'estructures de repetició.

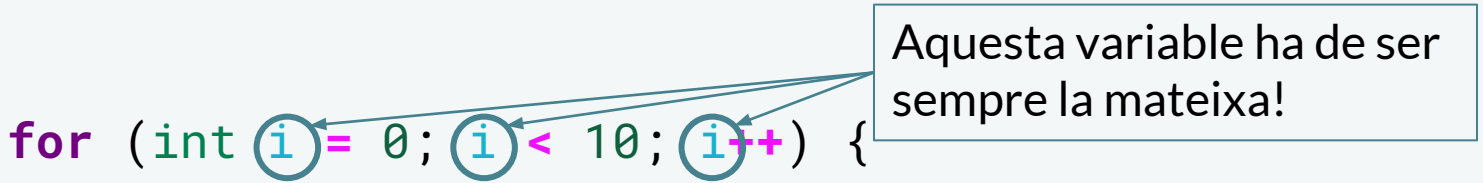
- Increment: es tracta d'una instrucció que modifica el valor del comptador, normalment una assignació. Aquesta instrucció s'executa automàticament al final de cada iteració.

2.- El bucle FOR

```
for (int i = 0; i < 10; i++) {

    System.out.println(i);

}
```



2.- El bucle FOR

- Imaginem que tenim el següent for. **Quin és l'àmbit de la variable i?**

```
for (int i = 0; i < 10; i++) {
    System.out.println(i);
}
```

Només té valor entre { i }

```
for (int i = 10; i < 20; i++) {
    System.out.println("Aprenem el bucle for");
}
```

Podem tenir un segon bucle utilitzant la mateixa variable de control.

2.- El bucle FOR



- **For 1:** Feu que es mostrin tots els números del 100 a l'1.

- **For 2:** Modifica l'exercici for 1 perquè pregunti a l'usuari el rang de número que vol mostrar.

- **For 3:** Mostrar per pantalla els números imparells de l'1 al 100 i digues quants n'hi ha en total.

2.- El bucle FOR



- **For 4:** Mostra la taula de multiplicar del número que t'introdueixi l'usuari en format. Exemple: 8 x 1 = 8…

- **For 5:** Fes un programa que calculi la mitjana de 5 números que introdueixi l'usuari. Utilitza el bucle per demanar els números a l'usuari.

- **For 6:** Calcula l'àrea de la circumferència de tots els radis d'1 a 10.

$$a = \pi * r^2$$

2.- El bucle FOR

- **For Extra:** Volem un programa que compleixi el següent algorisme:
  - Demanar quants usuaris vol emmagatzemar.
  - Per cada usuari, assignar un número d'usuari automàticament (inventat per tu) i demanar les dades: edat i categoria.
  - Finalment, mostrar les dades en format:

    Num_usuari -> Categoria (Edat)

3.- El bucle WHILE

- La sentència while permet repetir l'execució del bucle mentre es verifiqui la condició lògica.
- Aquesta condició es verifica al principi de cada iteració.
- Si la primera vegada, tot just quan s'executa la sentència per primer cop, ja no es compleix, no s'executa cap iteració.

3.- El bucle WHILE

- Quan utilitzem un bucle, és molt possible que no sapiguem quantes vegades s'haurà d'executar i, per tant, no podem utilitzar un for.

- Per exemple: endevinar un número, introduir números fins que introdueixin un número negatiu, introduir opcions d'un menú fins a l'opció "sortir".

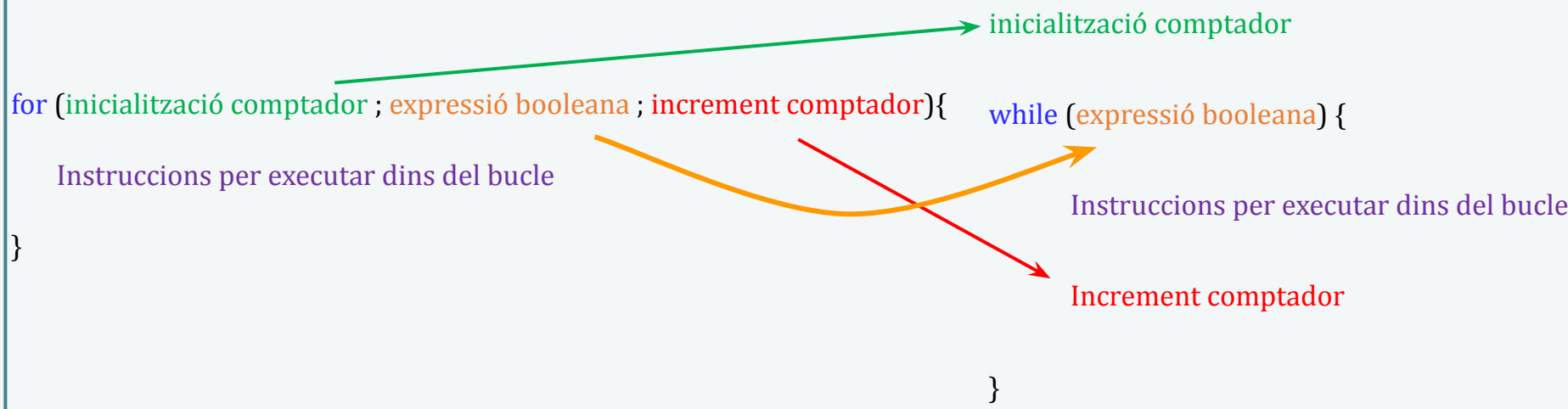
```
while (expressió booleana) {

    Instruccions per a executar dins del bucle

}
```

3.- El bucle WHILE

- De fet, la sentència for és equivalent a una sentència while que segueixi l'estructura següent:



3.- El bucle WHILE

- Les variables de control dins d'un bucle es poden englobar dins d'algun d'aquests tipus de comportament:
  - Comptador: Suma una quantitat constant a cada pas (de un en un, per exemple).
  - Acumulador: Suma una quantitat variable en cada iteració (`ex. suma = suma + preuProducte`)
  - Semàfor o bandera: una variable que serveix com a interruptor de si cal seguir fent iteracions. Habitualment es fa servir variables booleanes.

3.- El bucle WHILE

- Exemple.

```
int i=0;

while (i<10) {

    System.out.print("*");

   i++;

}
```

Quin tipus de bucle és?

Acumulador, semàfor o comptador?

Solució : Comptador

3.- El bucle WHILE

```
int i = 0;

while (i < 5) {

        System.out.println("Valor i - " + i);

        i++;

}
```

Comptador
Augmentem la variable de forma constant.

3.- El bucle WHILE

```
boolean semafor = true;
Scanner sc = new Scanner(System.in);
while (semafor == true) {
    System.out.println("Introdueix un valor:");
    int num = sc.nextInt();
     sc.nextLine();
     if (num < 0){
          semafor = false;
    }
}
```

Semàfor
Si el valor de num és negatiu, canviem la variable semàfor a false.

3.- El bucle WHILE

```
Scanner sc = new Scanner(System.in);
int num = 1;
while (num > 0) {
    System.out.println("Introdueix un valor:");
    num = sc.nextInt();
    sc.nextLine();
}
```

Semàfor
Podem simplificar el codi anterior amb la condició directa al bucle.

3.- El bucle WHILE

Acumulador
Augmentem la variable de valors diferents

```
Scanner sc = new Scanner(System.in);
double totalEstalviat = 0;

while (totalEstalviat < 100) {
     System.out.println("Estalvi: " + totalEstalviat + " €");
     System.out.print("Quants diners vols guardar a la guardiola?: ");
     double ingres = sc.nextDouble();
     sc.nextLine();
     totalEstalviat += ingres;
}

System.out.println("Tens un total de: " + totalEstalviat + " €");
```

3.- El bucle WHILE



- **While 1:** Escriure per pantalla una línia horitzontal a la pantalla exactament amb cent caràcters '-'.

- **While 2:** Modifica el *While 1* perquè pregunti a l'usuari quants caràcters '-' vol escriure per pantalla, i llavors els mostri.

- **While 3:** Feu un bucle que vagi des de 0 a un número que ens digui l'usuari per teclat. Dintre del bucle, mostra els números que són múltiples de 3.

3.- El bucle WHILE



- **While 4:** Feu un programa que permeti sumar tots els valors enters que vagi introduint l'usuari. El programa pararà quan l'usuari introdueixi un 0.

- **While 5:** Feu un programa en què cal endevinar un valor secret. Cal preguntar a l'usuari fins que l'encerti.

- **While 6:** Ampliar l'exercici 5 perquè hi hagi un màxim de tres intents.

3.- El bucle WHILE



- **Exercici extra:** Feu un programa que segueixi el següent algorisme:
    1. Demana per teclat un número (dada real).
    2. Comprova que el número és diferent de zero.
    3. En el cas que es compleixi: Calcula la meitat del número i mostra per pantalla el missatge: "La meitat de <número> és <meitat>".
    4. Repeteix els passos de l'1 al 3, sempre que el número introduït sigui diferent de zero.
    5. Mostra per pantalla el total de números diferents de zero que s'han introduït

4.- El bucle DO-WHILE

- A vegades, volem que un mateix codi es repeteixi un nombre finit de vegades, tot verificant una condició lògica, però volem que, com a mínim, s'executi 1 cop.

- Per a això utilitzem la sentència do...while.

- En aquesta estructura, la condició s'avalua després de la 1a execució del codi.

4.- El bucle DO-WHILE

```
do {
    Instruccions per a executar dins del bucle
    //Actualització del comptador, si hi ha
} while (expressió booleana);
```

4.- El bucle DO-WHILE

```
int i=0;

do{

    System.out.println("*");

    i++;

}while(i<10);
```

4.- El bucle DO-WHILE

- Quina diferència en la sortida hi ha entre aquests dos codis?

```
int i=0;
do{
    System.out.print("*");
    i++;
}while(i < 10);
```

```
int i=0;
while (i < 10) {
    System.out.print("*");
    i++;
}
```

CAP!

4.- El bucle DO-WHILE

- Quina diferència en la sortida hi ha entre aquests dos codis?

```
int i=0;

do{

    System.out.print("*");

    i++;

}while(i > 10);
```

```
int i=0;

while (i > 10) {

    System.out.print("*");

    i++;

}
```

Com no es compleix la condició, en el primer cas ens assegurem que s'executi un cop

4.- El bucle DO-WHILE

```
Scanner s = new Scanner(System.in);
System.out.println("1. Opció 1");
System.out.println("2. Opció 2");
System.out.println("3. Opció 3");
System.out.println("4. Sortir");
int opcio = s.nextInt();
s.nextLine();

while (opcio != 4) {
    switch (opcio) {
        case 1 -> System.out.println("Opció1 Escollida");
        case 2 -> System.out.println("Opció2 Escollida");
        case 3 -> System.out.println("Opció3 Escollida");
        case 4 -> System.out.println("Sortir");
    }
    System.out.println("1. Opció 1");
    System.out.println("2. Opció 2");
    System.out.println("3. Opció 3");
    System.out.println("4. Sortir");
    opcio = s.nextInt();
    s.nextLine();
}
```

Menú fet amb while

4.- El bucle DO-WHILE

```
int opcio = 0;
Scanner s = new Scanner(System.in);
do {
     System.out.println("1. Opció 1");
     System.out.println("2. Opció 2");
     System.out.println("3. Opció 3");
     System.out.println("4. Sortir");
     opcio = s.nextInt();
     s.nextLine();
     switch (opcio) {
         case 1 -> System.out.println("Opció1 Escollida");
         case 2 -> System.out.println("Opció2 Escollida");
         case 3 -> System.out.println("Opció3 Escollida");
         case 4 -> System.out.println("Sortir");
     }
 } while (opcio != 4);
```

Menú fet amb do...while

4.- El bucle DO-WHILE



- **DoWhile 1:** Feu un programa que faci un compte enrere fins a 0. El programa ha de demanar a l'usuari un nombre enter positiu on començar.

- **DoWhile 2:** Realitza un programa que demani una lletra. Si es prem qualsevol altra tecla que no sigui la "C", diu "lletra incorrecta" i torna a demanar una lletra. Quan es prem la tecla "C" el programa diu "gràcies" i acaba.

4.- El bucle DO-WHILE



- **DoWhile 3:** Feu un programa que demani números a l'usuari i s'aturi quan el número introduït sigui negatiu. El programa ha de mostrar quin és el número més gran que s'ha introduït.

- **DoWhile4:** Fes un programa que demana per teclat un nom d'usuari i després li demana confirmació. Si l'usuari diu que no, el procés es repeteix des del principi.

Crea el teu nom d'usuari: super_heroi

El nom "super_heroi" és correcte? (S/N): N

Crea el teu nom d'usuari: mega_crack

El nom "mega_crack" és correcte? (S/N): S

Nom d'usuari creat correctament!

4.- El bucle DO-WHILE



- **DoWhile extra:** Segueix el següent algorisme:
    1. Demanar la quantitat de producte a introduir. (Exemple: 3 bolígrafs)
    2. Demanar el preu d'un producte (Exemple: 1,20 €)
    3. Calcular el preu total del producte. (Exemple: 3 * 1,20)
    4. Sumar-ho al total de tots els productes.
    5. Repetir els passos 1-4, anar demanant més productes fins que el preu i la quantitat siguin 0.
    6. Mostrar el total de tots els productes.